

Introducción a La Programación Funcional con Kotlin

**UNTDF – Sede Ushuaia
2025**

Introducción a la programación funcional en Kotlin

“Un lenguaje funcional se basa en funciones puras, evaluación por sustitución, y evita estados mutables”

→ En Kotlin: no es obligatorio, pero **podemos elegir trabajar así** (como en Scheme). “Funciones como ciudadanos de primera clase, orden superior y composición son pilares del paradigma funcional”

“Un lenguaje funcional se basa en funciones puras, evaluación por sustitución, y evita estados mutables”

→ En Kotlin: no es obligatorio, pero **podemos elegir trabajar así** (como en Scheme).

“Funciones como ciudadanos de primera clase, orden superior y composición son pilares del paradigma funcional”

– *Cap. 11 de Gabbrielli-Martini, Cap. 15 de Sebesta.*

Paradigmas de programación y el lugar del funcional

En Scheme estábamos obligados a pensar funcionalmente. En Kotlin, elegimos cuándo hacerlo.

Paradigma	Modelo mental	Claves	Ejemplos
Imperativo	Paso a paso / recetas	Variables, asignaciones, bucles	C, Python, Kotlin
Orientado a objetos	Objetos que interactúan	Clases, métodos, herencia	Java, Kotlin
Funcional	Composición de funciones	Inmutabilidad, funciones puras	Scheme, Haskell
Lógico	Declaración de hechos	Reglas, resolución por búsqueda	Prolog

Repaso: Lo que ya sabemos con Scheme

- Pensamos en expresiones, no en instrucciones.
- Funciones puras, sin efectos colaterales.
- Inmutabilidad por defecto.
- Uso de funciones anónimas y de orden superior.
- Herramientas clave: apply, lambda, listas.define.
- Composición de funciones, recursión como control de flujo.

Scheme nos dio un laboratorio puro de programación funcional.

✓ Ejemplo completo: concatenación de dos listas en Scheme

```
scheme Copiar Editar  
  
(define (concatenar lst1 lst2)  
  (if (null? lst1)  
      lst2  
      (cons (car lst1) (concatenar (cdr lst1) lst2))))
```

✏ Ejemplo de uso:

```
scheme Copiar Editar  
  
(concatenar '(1 2 3) '(4 5 6))  
;; => (1 2 3 4 5 6)
```

Kotlin: un lenguaje multiparadigma

Kotlin no es un lenguaje funcional, pero permite aplicar los principios del paradigma funcional de forma natural y expresiva.

¿Qué toma Kotlin del paradigma funcional?

- **Funciones como valores**
Se pueden guardar, pasar y componer libremente.
- **Funciones de orden superior**
`map`, `filter`, `fold`, `partition`, `groupBy`, etc.
- **Lambdas y funciones anónimas**
Sintaxis clara para funciones declaradas en contexto.
- **Inmutabilidad**
`val`, estructuras inmutables (`listOf`, `mapOf`), sin asignación destructiva.
- **Composición de funciones**
Estilo encadenado: `list.map { ... }.filter { ... }.sum()`
- **Recursión terminal (tail recursion)**
Uso de `tailrec` para escribir funciones recursivas eficientes

```
tailrec fun factorial(n: Int, acc: Int = 1): Int =  
    if (n <= 1) acc else factorial(n - 1, n * acc)
```

Kotlin: un lenguaje multiparadigma

Kotlin **no impone** el paradigma funcional, pero lo **facilita elegantemente**. Nos permite elegir el enfoque adecuado según el problema y el estilo de programación que buscamos. Kotlin corresponde a una familia de lenguajes modernos que aprovechan lo mejor de cada paradigma.

Paradigma funcional

Inmutabilidad

Declaración > instrucción

Funciones puras

Composición de operaciones

Transparencia referencial

Paradigma imperativo / OOP

Control fino sobre estado y flujo

Secuencias de pasos claras

Estructuras modificables (mutable)

Patrones de diseño reutilizables

Optimización manual de performance

Kotlin: un lenguaje multiparadigma

- ◆ Podemos usar estilo funcional para procesamiento de datos (`map`, `filter`, `fold`)
- ◆ Usar `val` e inmutabilidad para mayor seguridad
- ◆ Usar `var` y clases cuando se necesita eficiencia, control o mutabilidad
- ◆ Encapsular lógicas funcionales dentro de objetos
- ◆ Modularidad: lo declarativo y lo imperativo conviven sin conflicto

```
val resultado = datos
    .filter { it.activo }
    .map { it.valor * 2 }
    .sum()
```

Representar datos con funciones

En Kotlin:

- Podemos **guardar funciones en variables**
- **Pasarlas como argumentos**
- **Retornar funciones como resultados**

```
val cuadrado = { x: Int -> x * x }  
fun aplicar(f: (Int) -> Int, x: Int) = f(x)  
aplicar(cuadrado, 5) // 25
```

Todo esto está respaldado por el sistema de tipos estático de Kotlin.

► **Problema para investigar:** Escribir una función que devuelva otra función. ¿En qué caso real podría ser útil?

¿Qué es val en Kotlin?

En Kotlin, `val` se usa para declarar una **referencia inmutable**: una vez asignado un valor, **no se puede volver a asignar**.

Ejemplo:

```
val x = 10
```

```
x = 20 // ❌ Error: no se puede reasignar una variable `val`
```

¿Immutable significa constante?

No exactamente. `val` no hace que el **contenido** del objeto sea inmutable, sólo que **la referencia no puede cambiar**.

```
val lista = mutableListOf(1, 2, 3)
```

```
lista.add(4) // ✅ permitido
```

```
lista = mutableListOf(5, 6) // ❌ no permitido
```

val vs var

Declaración	¿Reasignable?	¿Contenido mutable?	Uso típico
val	× No	✅ Puede serlo	Estilo funcional
var	✅ Sí	✅ Puede serlo	Estilo imperativo

Lambdas y sintaxis flexible

- Lambdas simples:

```
val suma = { a: Int, b: Int -> a + b }
```

- Último argumento lambda fuera de paréntesis:

```
list.filter { it > 0 }
```

- Lambda como función anónima:

```
val multiplicar = fun(x: Int, y: Int): Int = x * y
```

Kotlin permite escribir código expresivo sin sacrificar legibilidad.

► **Problema para investigar:** Comparar sintaxis de lambdas entre Kotlin, Python y JavaScript. ¿Cuál resulta más clara?

colecciones en Kotlin

Kotlin provee colecciones estándar listas para usar:

- **Listas (List)**: ordenadas, con acceso por índice
- **Conjuntos (Set)**: sin elementos duplicados
- **Mapas (Map)**: pares clave–valor

```
val lista = listOf(1, 2, 3)
```

```
val conjunto = setOf("A", "B", "C")
```

```
val mapa = mapOf("a" to 1, "b" to 2)
```

Las colecciones son fundamentales para trabajar con datos de forma declarativa.

► **Problema para investigar:** ¿Qué diferencia hay entre `listOf` y `mutableListOf`?

colecciones en Kotlin

Colecciones tienen muchas operaciones útiles:

```
val lista = listOf(1, 2, 3, 4)

println(lista.first())    // 1
println(lista.last())     // 4
println(lista.size)       // 4
println(lista.contains(3)) // true
```

Estas funciones ya nos permiten explorar y consultar datos fácilmente.

► **Problema para investigar:** ¿Qué hace `list.indexOf(elemento)`?

Iteración clásica y funcional

- Forma imperativa:

```
for (n in lista) {  
    println(n)  
}
```

- Forma funcional:

```
lista.forEach { println(it) }
```

Esta diferencia marca el paso del paradigma imperativo al funcional.

► **Problema para investigar:** Comparar `forEach` con `map`. ¿En qué se diferencian?

Inmutables vs Mutables

Kotlin distingue entre colecciones **inmutables** y **mutables**:

Tipo	Modificable	Ejemplo
<code>listOf()</code>	× No	<code>val l = listOf(1)</code>
<code>mutableListOf()</code>	✓ Sí	<code>val l = mutableListOf(1)</code>

```
val nums = mutableListOf(1, 2)
```

```
nums.add(3) // ok
```

Usar inmutabilidad favorece el estilo funcional.

► Problema para investigar: ¿Por qué Kotlin no impide modificar una colección mutable aunque esté declarada con `val`?

Funciones de orden superior

Funciones que **reciben o devuelven funciones**:

```
fun operar(op: (Int, Int) -> Int, a: Int, b: Int) = op(a, b)  
  
operar({ x, y -> x * y }, 3, 4) // 12
```

Ya usamos esto todo el tiempo con:

- `map`, `filter`, `reduce`, `fold`, `partition`, `groupBy`

La programación funcional en Kotlin comienza en las colecciones.

► **Problema para investigar:** Escribir una función `transformarLista` que reciba una lista y una operación (función) y la aplique.

Tail

Kotlin permite optimizar recursiones terminales:

```
tailrec fun factorial(n: Int, acc: Int = 1): Int =  
    if (n <= 1) acc else factorial(n - 1, acc * n)
```

Evita desbordes de pila y permite escribir funciones sin bucles.

► **Problema para investigar:** Escribir una función `tailrec` que calcule el n-ésimo número de Fibonacci.

Encadenamiento y composición

Transformación fluida de datos:

```
val resultado = lista  
    .filter { it > 0 }  
    .map { it * 2 }  
    .sum()
```

Se pueden usar `asSequence()` para trabajar con evaluación perezosa.

Se prioriza claridad, declaratividad y composición.

► **Problema para investigar:** Reescribir la expresión anterior usando `asSequence()` y verificar cuándo conviene usarlo.

map y filter: transformación funcional de listas

Las funciones `map` y `filter` son la base de la transformación declarativa de colecciones.

```
val numeros = listOf(1, 2, 3, 4, 5, 6)
```

```
val pares = numeros.filter { it % 2 == 0 }    // [2, 4, 6]
```

```
val cuadrados = numeros.map { it * it }       // [1, 4, 9, 16, 25, 36]
```

Se pueden encadenar:

```
val resultado = numeros
```

```
    .filter { it > 3 }
```

```
    .map { it * 10 }    // [40, 50, 60]
```

El código expresa **qué queremos** sin detallar cómo hacerlo.

► **Problema para investigar:** ¿Cómo reescribirías este código con bucles imperativos? ¿Cuál es más claro?

fold y reduce: acumuladores funcionales

Estas funciones permiten **reducir** una colección a un solo valor:

```
val suma = listOf(1, 2, 3).reduce { acc, n -> acc + n } // 6
```

```
val factorial = listOf(1, 2, 3, 4).fold(1) { acc, n -> acc * n } // 24
```

Diferencia clave:

- **reduce**: usa el **primer elemento** como acumulador inicial
- **fold**: permite **especificar un valor inicial**

```
val letras = listOf("a", "b", "c").fold("→") { acc, letra -> "$acc $letra" }  
  
// → a b c
```

► **Problema para investigar**: ¿Por qué **fold** es más seguro que **reduce** en listas vacías

groupBy y partition: reorganizar colecciones

Agrupan elementos según una clave:

```
val personas = listOf("Ana", "Axel", "Bea", "Bruno")  
  
val agrupado = personas.groupBy { it.first() }  
  
// {A=[Ana, Axel], B=[Bea, Bruno]}
```

Parten en dos listas:

```
val (positivos, negativos) = listOf(-3, -2, 5, 6).partition { it > 0 }  
  
// positivos = [5, 6], negativos = [-3, -2]
```

Herramientas poderosas para categorizar y filtrar datos.

► **Problema para investigar:** ¿Cómo usarías `groupBy` para agrupar palabras por su longitud?

asSequence

```
val resultado = (1..1_000_000)

    .filter { it % 2 == 0 }

    .map { it * 2 }

    .take(5) // Toma los primeros 5
```

Sin `asSequence()`, filtra y mapea todos los millones de números,
aunque solo queríamos los primeros 5.

Con `asSequence()`:

```
val resultado = (1..1_000_000).asSequence()

    .filter { it % 2 == 0 }

    .map { it * 2 }

    .take(5)

    .toList() // convierte en lista
```

► **Problema para investigar:** Medir diferencia de tiempo con y sin `asSequence()` usando `System.currentTimeMillis()`.

flatMap: aplanar y transformar

Cuando cada elemento se transforma en una colección, `flatMap` permite aplanar el resultado.

```
val lineas = listOf("uno dos", "tres cuatro")
```

```
val palabras = lineas.flatMap { it.split(" ") }
```

```
// [uno, dos, tres, cuatro]
```

Combina `map` + `flatten` en una sola operación eficiente.

🧠 Ideal para transformar listas de listas, o resultados encadenados.

► **Problema para investigar:** ¿Qué sucede si usás `map` en vez de `flatMap` en el ejemplo anterior?

zip: combinar listas en pares

Une dos listas en una sola colección de pares:

```
val nombres = listOf("Ana", "Beto", "Caro")
```

```
val edades = listOf(21, 30, 27)
```

```
val personas = nombres.zip(edades)
```

```
// [(Ana, 21), (Beto, 30), (Caro, 27)]
```

También se puede usar con una transformación:

```
val personasStr = nombres.zip(edades) { n, e -> "$n tiene $e años" }
```

► **Problema para investigar:** ¿Qué pasa si las listas tienen tamaños distintos?

Librerías y frameworks para programación funcional

◆ 1. Arrow

La librería funcional más completa para Kotlin.

Provee tipos como `Either`, `Option`, `Validated`, `Try`, y abstracciones como `Functor`, `Applicative`, `Monad`, `raise.either`, etc.

◆ 2. Kotlinx.coroutines

Concurrencia funcional con coroutines.

Permite flujos inmutables (`Flow`, `StateFlow`, `SharedFlow`) y composición estructurada de asincronía sin callbacks.

◆ 3. Reactor Kotlin Extensions / RxKotlin

Extensiones reactivas para flujos y transformación de datos en tiempo real.

Enfocado en flujos continuos (`Flux`, `Mono`) y composición de pipelines funcionales.

◆ 4. Kotest (antes KotlinTest)

Librería para testing funcional y declarativo.

Soporta propiedades (`property testing`), matchers expresivos y testing puro sin mocks imperativos.

Estas herramientas permiten extender la programación funcional más allá de las colecciones: hacia validación, concurrencia, lógica de dominio y testing.

¿Qué es Arrow?

Arrow es una biblioteca funcional para Kotlin que proporciona herramientas y abstracciones para aplicar conceptos de programación funcional en proyectos reales. Su objetivo es complementar el lenguaje Kotlin con estructuras y patrones funcionales que permiten escribir código más seguro, expresivo y mantenible.

Características principales:

- **Tipos de datos funcionales:** `Option`, `Either`, `Validated`, `Try`, entre otros.
- **Manejo de efectos:** Control de efectos secundarios y concurrencia mediante coroutines.
- **Optics:** Manipulación de estructuras de datos inmutables de forma segura y concisa.
- **Resiliencia:** Patrones para construir aplicaciones robustas y tolerantes a fallos.

¿Qué es Arrow?

Aunque Kotlin ofrece soporte para programación funcional, Arrow amplía estas capacidades y proporciona herramientas adicionales que facilitan la adopción de este paradigma.

Ventajas de usar Arrow:

- **Manejo seguro de errores:** Evita el uso de excepciones mediante tipos como `Either` y `Validated`.
- **Código más expresivo:** Utiliza combinadores funcionales (`map`, `flatMap`, `fold`) para transformar y componer operaciones.
- **Inmutabilidad:** Fomenta el uso de estructuras de datos inmutables, reduciendo efectos secundarios.
- **Composición de funciones:** Facilita la creación de funciones modulares y reutilizables.

¿Qué es Arrow?

Arrow proporciona varios tipos de datos que modelan diferentes escenarios en la programación funcional.

1. **Option:**

- Representa un valor que puede estar presente (**Some**) o ausente (**None**).
- Evita el uso de valores nulos y las excepciones asociadas.

2. **Either:**

- Modela una operación que puede resultar en un valor exitoso (**Right**) o en un error (**Left**).
- Útil para el manejo de errores sin lanzar excepciones.

3. **Validated:**

- Permite la acumulación de múltiples errores en lugar de fallar en el primer error.
- Ideal para validaciones de formularios o entradas de usuario.

4. **Try:**

- Captura excepciones en operaciones que pueden fallar, devolviendo un resultado exitoso o un error.